

Para este eje final, que es eminentemente práctico, en lugar de un texto teórico denso, prepararé una **Guía Práctica Detallada**. Esta guía servirá como un "paso a paso" estructurado para construir una aplicación web MVC completa pero sencilla, integrando todos los conceptos que hemos visto. No será un simple "copiar y pegar" para *cualquier* proyecto (porque cada proyecto tendrá sus particularidades y el objetivo es que apliquen lo aprendido), pero sí proporcionará ejemplos concretos y el flujo de trabajo para una aplicación típica, que podrán usar como base y referencia sólida para su propio Proyecto Integrador Final.

Recuerden que **esta guía es un ejemplo y una hoja de ruta**. Deberán adaptar los nombres de clases, propiedades, lógica y funcionalidades a los requisitos específicos de *su* proyecto. El objetivo es entender el *proceso* y cómo se conectan las piezas. **El uso continuo de Git es fundamental en cada paso.**

Guía Práctica de Estudio: Construyendo una Aplicación Web MVC Completa

Profesor: Jorge Izaguirre

Materia: Programación .NET

Curso: 6to Año - TÉCNICO EN PROGRAMACIÓN

Eje Temático: 10 - Proyecto Integrador Final

Introducción

Esta guía te acompañará en el proceso de construir una aplicación web ASP.NET Core MVC desde cero hasta un producto funcional básico, aplicando los conocimientos adquiridos en la materia. Usaremos como ejemplo una aplicación simple para gestionar un **Catálogo de Productos** agrupados por **Categorías**.

Paso 0: Prerrequisitos y Herramientas

Asegúrate de tener instalado y funcionando:

- Visual Studio 2022 (o una versión reciente) con la carga de trabajo "Desarrollo de ASP.NET y web".
- .NET SDK (versión 6, 7, 8 o la que estén usando en clase).
- SQL Server Express (o Developer Edition).
- SQL Server Management Studio (SSMS) o Azure Data Studio.
- Git.
- Una cuenta de GitHub.

Paso 1: Definición y Planificación (Ejemplo: Catálogo Productos)

- **Objetivo:** Crear una web para listar, ver detalles, añadir, editar y eliminar productos. Los productos pertenecerán a una categoría.
- **Entidades Principales:**
 - *Categoría*: Id (PK, int), Nombre (string, requerido).
 - *Producto*: Id (PK, int), Nombre (string, requerido), Descripción (string, opcional), Precio (decimal, requerido, >0), Stock (int, requerido, >=0), Activo (bool), CategoricalId (int, FK a Categoría).
- **Relación:** *Categoría* (1) ---< *Producto* (N) (Una categoría tiene muchos productos, un producto pertenece a una categoría).
- **Funcionalidades Clave (CRUD):**
 - Gestionar Categorías (Listar, Crear, Editar, Eliminar).
 - Gestionar Productos (Listar, Crear, Editar, Eliminar, Ver Detalles).

- Mostrar el nombre de la categoría al listar/ver productos.
- Poder seleccionar la categoría al crear/editar un producto.
- **Diseño BD (Conceptual):** Dos tablas: **Categorías** y **Productos**, con las columnas mencionadas y una FK **CategoriaId** en **Productos** apuntando a **Id** en **Categorías**.

Paso 2: Creación del Proyecto y Configuración de Git

1. **Crear Proyecto MVC:** En Visual Studio, **Archivo > Nuevo > Proyecto > Aplicación web ASP.NET Core (Modelo-Vista-Controlador)**. Nómbralo (ej: **CatalogoWebMVC**), elige ubicación, selecciona la versión de .NET. **Autenticación: Ninguna** (por simplicidad en este ejemplo). Asegúrate que esté marcado "Configurar para HTTPS". Crear.
2. **Inicializar Git:**
 - Abre una terminal (Git Bash, PowerShell, CMD) en la **carpeta raíz** de la solución creada.
 - Ejecuta: `git init`
 - Crea un archivo `.gitignore` en la raíz (puedes copiar uno estándar para Visual Studio/.NET de [gitignore.io](https://github.com/gitignore.io)). Es crucial para ignorar `bin/`, `obj/`, archivos de usuario, etc.
 - Ejecuta: `git add .`
 - Ejecuta: `git commit -m "Initial project setup with .gitignore"`
3. **Crear Repositorio Remoto en GitHub:**
 - Ve a tu cuenta de GitHub y crea un nuevo repositorio (ej: **catalogo-web-mvc**), **sin** inicializarlo con README, .gitignore o licencia.
 - Copia la URL HTTPS del repositorio creado.
4. **Conectar Local con Remoto:**
 - En la terminal de tu proyecto: `git remote add origin URL_DE_GITHUB_COPIADA`
 - Verifica con: `git remote -v`
 - Sube el primer commit: `git push -u origin main` (o `master` si esa es tu rama por defecto).

Paso 3: Creación de Modelos (Entidades POCO) y DbContext

1. **Crear Clases Modelo:** En la carpeta **Models**, crea las clases POCO.
 - **Models/Categoria.cs:**

```
using System.ComponentModel.DataAnnotations; // Necesario para
DataAnnotations

namespace CatalogoWebMVC.Models
{
    public class Categoria
    {
        public int Id { get; set; } // Convención para PK

        [Required(ErrorMessage = "El nombre de la categoría es obligatorio.")]
        [StringLength(50, ErrorMessage = "El nombre no puede exceder los 50 caracteres.")]
        [Display(Name = "Nombre de Categoría")]
        public string Nombre { get; set; }

        // Propiedad de navegación: Una categoría tiene muchos
    }
}
```

```

productos
    public virtual ICollection<Producto> Productos { get; set; } =
new List<Producto>();
    }
}

```

- Models/Producto.cs:

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema; // Para [Column]

namespace CatalogoWebMVC.Models
{
    public class Producto
    {
        public int Id { get; set; } // PK

        [Required(ErrorMessage = "El nombre es obligatorio.")]
        [StringLength(100)]
        public string Nombre { get; set; }

        [StringLength(500)]
        [DataType(DataType.MultilineText)] // Ayuda a Tag Helpers
        public string Descripcion { get; set; }

        [Required(ErrorMessage = "El precio es obligatorio.")]
        [Column(TypeName = "decimal(18,2)")] // Especificar tipo SQL
para dinero
        [Range(0.01, 1000000.00, ErrorMessage = "El precio debe ser
positivo.")]
        [DataType(DataType.Currency)]
        public decimal Precio { get; set; }

        [Required(ErrorMessage = "El stock es obligatorio.")]
        [Range(0, int.MaxValue, ErrorMessage = "El stock no puede ser
negativo.")]
        public int Stock { get; set; }

        public bool Activo { get; set; } = true; // Valor por defecto

        // --- Clave Foránea y Navegación ---
        [Required(ErrorMessage = "Debe seleccionar una categoría.")]
        [Display(Name = "Categoría")]
        public int CategoriaId { get; set; } // FK a la tabla
Categorias

        // Propiedad de navegación: Un producto pertenece a UNA
categoría
        [ForeignKey("CategoriaId")] // Vincula explícitamente la FK con
la navegación
        public virtual Categoria Categoria { get; set; }
    }
}

```

```
}
}
```

2. **Crear DbContext:** Crea una carpeta **Data** en la raíz del proyecto. Dentro, crea **ApplicationDbContext.cs**.

```
using Microsoft.EntityFrameworkCore;
using CatalogoWebMVC.Models; // Namespace de tus modelos

namespace CatalogoWebMVC.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
            : base(options)
        {
        }

        // DbSet para cada entidad que EF Core debe gestionar (se mapearán a
tablas)
        public DbSet<Categoria> Categorias { get; set; }
        public DbSet<Producto> Productos { get; set; }

        // Aquí podrías usar OnModelCreating para configuración Fluent API
si fuera necesario
        // protected override void OnModelCreating(ModelBuilder
modelBuilder) { ... }
    }
}
```

3. **Comitear:** `git add ., git commit -m "Add Categoria and Producto models and ApplicationDbContext"`

Paso 4: Configuración de EF Core y Primera Migración

1. **Cadena de Conexión:** Abre **appsettings.json** y añade la sección **ConnectionStrings**.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost\\SQLEXPRESS;Database=CatalogoMVC_DB;Trusted_Connection=Tru
e;MultipleActiveResultSets=true;TrustServerCertificate=True"
  },
  "Logging": { /*...*/ },
  "AllowedHosts": "*"
}
```

- **¡Importante!** Ajusta **Server** y **Database** a tu configuración.
- 2. **Instalar Paquetes NuGet:** Abre la Consola del Administrador de Paquetes (PMC: **Herramientas > Admin. Paquetes NuGet > Consola...**). Asegúrate que el "Proyecto predeterminado" sea tu proyecto MVC (**CatalogoWebMVC**). Ejecuta:
 - `Install-Package Microsoft.EntityFrameworkCore.SqlServer`
 - `Install-Package Microsoft.EntityFrameworkCore.Tools`
- 3. **Registrar DbContext en Program.cs (DI):**

```
// Al inicio de Program.cs
using Microsoft.EntityFrameworkCore;
using CatalogoWebMVC.Data; // Namespace del DbContext

var builder = WebApplication.CreateBuilder(args);

// --- Add services to the container. ---
var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString)); // Registrar DbContext

builder.Services.AddControllersWithViews();

var app = builder.Build();
// --- Configure the HTTP request pipeline. ---
// ... resto de la configuración ...
app.Run();
```

- 4. **Crear la Primera Migración:** En la PMC, ejecuta:
 - `Add-Migration InitialCreate`
 - Revisa el archivo `.cs` generado en la nueva carpeta **Migrations/**. Debería contener código para crear las tablas **Categorias** y **Productos** con sus columnas, PKs y FKs.
- 5. **Aplicar Migración a la Base de Datos:** En la PMC, ejecuta:
 - `Update-Database`
 - Esto creará la base de datos **CatalogoMVC_DB** (si no existe) y las tablas definidas en la migración.
- 6. **Verificar:** Usa SSMS o Azure Data Studio para conectarte a tu instancia SQL Server y verifica que la base de datos y las tablas se hayan creado correctamente.
- 7. **Commitear:** `git add ., git commit -m "Configure EF Core, DI, and run initial migration", git push`

Paso 5: Implementando CRUD para Categorías (Ejemplo Rápido)

(Para Productos será similar pero más detallado en el siguiente paso)

- 1. **Crear Controlador:** Clic derecho en **Controllers > Agregar > Controlador... > Controlador de MVC - Vacío**. Nómbralo **CategoriasController.cs**.
- 2. **Injectar DbContext:**

```

using CatalogoWebMVC.Data;
using CatalogoWebMVC.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore; // ParaToListAsync, etc.

public class CategoriasController : Controller
{
    private readonly ApplicationDbContext _context;
    private readonly ILogger<CategoriasController> _logger; // Añadir logger

    public CategoriasController(ApplicationDbContext context,
        ILogger<CategoriasController> logger)
    {
        _context = context;
        _logger = logger;
    }

    // GET: Categorias
    public async Task<IActionResult> Index()
    {
        _logger.LogInformation("Obteniendo lista de categorías...");
        var categorias = await _context.Categorias.OrderBy(c =>
c.Nombre).ToListAsync();
        return View(categorias); // Pasa la lista a la vista
    }

    // --- Implementarías acciones GET/POST para Create, Edit, Delete de
    forma similar ---
    // GET: Categorias/Create
    public IActionResult Create()
    {
        return View(); // Muestra el formulario vacío
    }

    // POST: Categorias/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create([Bind("Nombre")] Categoria
categoria) // [Bind] es una forma de especificar qué propiedades aceptar
    {
        if (ModelState.IsValid)
        {
            _context.Add(categoria);
            await _context.SaveChangesAsync();
            TempData["MensajeExito"] = "Categoría creada exitosamente.";
            return RedirectToAction(nameof(Index));
        }
        return View(categoria); // Vuelve a mostrar el form con errores
    }

    // ... (Edit GET/POST, Delete GET/POST similares) ...
}

```

3. **Crear Vistas:** Clic derecho en **Views > Agregar Carpeta > Categorías**. Luego clic derecho en **Views/Categorías > Agregar > Vista... > Vista Razor**. Crear:

- **Index.cshtml:**

```
@model IEnumerable<CatalogoWebMVC.Models.Categoria>
@{ ViewData["Title"] = "Categorías"; }

<h1>Categorías</h1>
<p><a asp-action="Create" class="btn btn-primary">Crear Nueva</a></p>

@if (TempData["MensajeExito"] != null) { /* Mostrar alert éxito */ }

<table class="table">
    <thead><tr><th>@Html.DisplayNameFor(model => model.Nombre)</th><th>
</th></tr></thead>
    <tbody>
        @foreach (var item in Model) {
            <tr>
                <td>@Html.DisplayFor(modelItem => item.Nombre)</td>
                <td>
                    @*<a asp-action="Edit" asp-route-
id="@item.Id">Editar</a> |
                    <a asp-action="Details" asp-route-
id="@item.Id">Detalles</a> |
                    <a asp-action="Delete" asp-route-
id="@item.Id">Eliminar</a>*</td>
            </tr>
        }
    </tbody>
</table>
```

- **Create.cshtml:**

```
@model CatalogoWebMVC.Models.Categoria
@{ ViewData["Title"] = "Crear Categoría"; }

<h1>Crear Categoría</h1>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="Create">
            <div asp-validation-summary="ModelOnly" class="text-
danger"></div>
            <div class="form-group mb-3"> <!-- mb-3 es margen bootstrap
--%>
                <label asp-for="Nombre" class="control-label"></label>
                <input asp-for="Nombre" class="form-control" />
                <span asp-validation-for="Nombre" class="text-danger">
</span>
```

```

        </div>
        <div class="form-group">
            <button type="submit" class="btn btn-
success">Crear</button> |
            <a asp-action="Index">Volver a la Lista</a>
        </div>
    </form>
</div>
</div>
@section Scripts { @{await
Html.RenderPartialAsync("_ValidationScriptsPartial");} }

```

- (Crearías vistas similares para Edit, Delete, Details).
- 4. **Añadir enlace en Layout:** En `Views/Shared/_Layout.cshtml`, añade un enlace a la acción Index de Categorías en la barra de navegación.
- 5. **Probar:** Ejecuta (F5), navega a Categorías, prueba crear una categoría. Verifica la validación.
- 6. **Commitear:** `git add ., git commit -m "Implement basic CRUD for Categorías", git push`

Paso 6: Implementando CRUD para Productos (Más Detallado)

Este paso es similar al anterior pero involucra la relación con Categorías y ViewModels.

1. Crear ViewModels (en carpeta `ViewModels`):

- `ProductoListViewModel.cs`: Para mostrar en la lista (quizás solo Id, Nombre, Precio, NombreCategoría).
- `ProductoViewModel.cs`: Para los formularios Create/Edit (con todas las propiedades necesarias, incluyendo `CategoriaId` y una `List<SelectListItem>` para el dropdown).

```

// Ejemplo ProductoViewModel.cs
using Microsoft.AspNetCore.Mvc.Rendering; // Para SelectListItem
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

public class ProductoViewModel
{
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio.")]
    [StringLength(100)]
    public string Nombre { get; set; }

    [StringLength(500)]
    [DataType(DataType.MultilineText)]
    public string Descripcion { get; set; }

    [Required(ErrorMessage = "El precio es obligatorio.")]
    [Column(TypeName = "decimal(18,2)")]
    [Range(0.01, 1000000.00)]
    [DataType(DataType.Currency)]
    public decimal Precio { get; set; }
}

```

```

[Required]
[Range(0, int.MaxValue)]
public int Stock { get; set; }

public bool Activo { get; set; }

[Required(ErrorMessage = "Debe seleccionar una categoría.")]
[Display(Name = "Categoría")]
public int CategoriaId { get; set; }

// Para llenar el DropDownList en la vista
public IEnumerable CategoriasList { get; set; }
}

```

2. **Crear `ProductosController.cs`**: Inyectar `ApplicationDbContext` y `ILogger`.

3. **Implementar Acción `Index (GET)`**:

- Query LINQ: `_context.Productos.Include(p => p.Categoria).Where(...).OrderBy(...).Select(p => new ProductoListViewModel { ... })`. Usar `Include` para traer el nombre de la categoría. Usar `Select` para mapear al ViewModel de lista.
- Pasar `List<ProductoListViewModel>` a la vista `Index.cshtml`.

4. **Implementar Acción `Create (GET)`**:

- Crear un `ProductoViewModel` vacío.
- **Cargar la lista de categorías para el dropdown:**

```

viewModel.CategoriasList = new SelectList(await
_context.Categorias.OrderBy(c=>c.Nombre).ToListAsync(), "Id",
"Nombre");

```

- Retornar `View(viewModel)`.

5. **Implementar Acción `Create (POST)`**:

- Recibir `ProductoViewModel viewModel`.
- Verificar `if (ModelState.IsValid)`.
- **Mapear** de `viewModel` a una nueva entidad `Producto`.
- `_context.Add(nuevoProducto);`
- `await _context.SaveChangesAsync;`
- `TempData[...] = "...";`
- `return RedirectToAction(nameof(Index));`
- Si no es válido: **¡Volver a cargar `viewModel.CategoriasList` como en el GET!** `return View(viewModel);`.

6. **Implementar Acciones `Edit (GET y POST)`**: Similar a `Create`, pero el `GET` busca el producto por `id`, mapea a `ProductoViewModel`, carga la lista de categorías y pasa el `viewModel` a la vista. El `POST` busca la entidad existente, actualiza sus propiedades desde el `viewModel`, y guarda los cambios.

7. **Implementar Acciones `Details` y `Delete (GET y POST)`**: Similar, mostrando detalles (incluyendo categoría) o confirmando eliminación.

8. **Crear Vistas (`Index`, `Create`, `Edit`, `Details`, `Delete`) en `Views/Productos/`:**

- Usar `@model` apropiado (ViewModel).
 - Usar Tag Helpers (`form`, `label`, `input`, `textarea`, `select asp-for="CategoriaId" asp-items="Model.CategoriasList"`, `span asp-validation-for`, `div asp-validation-summary`).
 - Asegurarse de incluir `@section Scripts { <partial name="_ValidationScriptsPartial" /> }` en las vistas con formularios.
9. **Probar CRUD Completo:** Crear, listar, ver detalles, editar (cambiando categoría), eliminar productos. Verificar validaciones.
10. **Commitear:** Hacer commits frecuentes durante este proceso. Ej: `"Implement ProductosController Index action and view"`, `"Implement Create actions and view for Productos"`, `"Add Category dropdown to Producto forms"`, etc. `git push`.

Paso 7: Refinamiento y Temas Adicionales (Integración)

- **Mejorar Vistas:** Aplicar estilos CSS básicos (puedes usar Bootstrap que viene por defecto). Usar *Layout*. Quizás crear alguna *Vista Parcial* si hay elementos repetidos.
- **Añadir Logging:** Incluir `ILogger` y añadir logs informativos o de error en puntos clave del controlador.
- **Usar TempData:** Asegurarse de dar feedback al usuario después de operaciones exitosas (Create, Edit, Delete).
- **(Opcional Avanzado):** Implementar el Patrón Repositorio para desacoplar el acceso a datos de los controladores.
- **(Opcional Avanzado):** Añadir Autenticación/Autorización básica usando Identity si se cubrió y el tiempo lo permite.

Paso 8: Testing Final

- Probar **exhaustivamente** todas las funcionalidades.
- Probar casos borde y datos inválidos.
- Corregir bugs.

Paso 9: Documentación

- Escribir un archivo `README.md` (o PDF) simple que incluya:
 - Nombre del proyecto y autores.
 - Descripción breve del propósito.
 - Instrucciones básicas para ejecutarlo (clonar, `Update-Database`, F5).
 - (Opcional) Diagrama ER de la base de datos.

Paso 10: Preparación de la Presentación

- Organizar una demo fluida de las funcionalidades principales.
- Preparar una breve explicación técnica (arquitectura MVC, EF Core, decisiones clave).
- Anticipar posibles preguntas.

Paso 11: ¡Presentación y Entrega!

- Realizar la presentación y demo.
- Entregar el link al repositorio de GitHub completo, incluyendo el código fuente, la carpeta `Migrations` y la documentación.

¡Importante! Esta guía es un esqueleto. Deberás rellenar la lógica específica, manejar errores potenciales (ej: `FindAsync` devuelve null), y adaptar todo a los requisitos particulares de tu proyecto. **¡Commitea a Git después de cada paso funcional o cambio significativo!**

Síntesis de una oración: Esta guía práctica detalla los pasos para desarrollar una aplicación web ASP.NET Core MVC completa con EF Core Code-First, desde la configuración inicial del proyecto y la base de datos con migraciones, hasta la implementación del CRUD con ViewModels, Tag Helpers, validación, y la gestión del proceso con Git, sirviendo como base para el proyecto integrador final.

Síntesis de 5 puntos importantes:

1. **Planificación y Setup:** Definir entidades/relaciones, configurar el proyecto MVC y el repositorio Git/GitHub inicial.
2. **Modelo y BD (Code-First):** Crear clases POCO (Entidades), `DbContext` con `DbSets`, configurar DI y usar Migrations (`add`, `update`) para generar/actualizar el esquema de la base de datos.
3. **CRUD con MVC/EF Core:** Implementar acciones GET/POST en Controladores (inyectando `DbContext`), usar LINQ (`Include`, `Select`, `Where`, etc.) para consultas, y EF Core (`AddAsync`, `FindAsync`, `Remove`, `SaveChangesAsync`) para CUD, pasando datos a Vistas fuertemente tipadas (`@model`) mediante ViewModels.
4. **Formularios y Validación:** Utilizar Tag Helpers (`asp-for`, `asp-action`, `asp-validation-for`, etc.) para generar formularios vinculados a ViewModels, e implementar validación con DataAnnotations y `ModelState.IsValid`.
5. **Proceso Iterativo y Entrega:** Desarrollar incrementalmente, probar, refinar, documentar brevemente y usar Git de forma constante para versionar el progreso antes de la presentación final.

60530